Digital VLSI Design Assignment

4-bit Array Multiplier from RTL to GDSII (DVD Gladiators)

Ramit Nag Roll No: 2026203011
Kanuthala Harshith Roll No: 2026203018

1 Introduction

A 4-bit unsigned array multiplier (16 AND gates, 12 full adders, 16 flip-flops, latency 2 clock cycles) was written in Verilog, verified exhaustively, synthesized with Yosys, timed with OpenSTA and taken to a DRC- and LVS-clean GDSII with LibreLane on SkyWater SKY130 (`sky130_fd_sc_hd`). All work was done in the IIC-OSIC-TOOLS Docker image. Repository: https://github.com/Harshith21k/mult4_2026203018.

AI-tool disclosure. An AI assistant (Claude) was used for guidance on the tool flow, debugging and for drafting this report. All code in the repository was written and checked by us.

2 Design and Verification (Part A)

The design registers a and b (8 flip-flops), builds the array from the registered values (row 0 only ANDs; rows 1–3 each 4 AND + 4 full adders in a ripple chain), and registers the 8-bit product (8 flip-flops). Reset is asynchronous and active low. Fig. 1 shows the register stages and Fig. 2 shows the array itself.

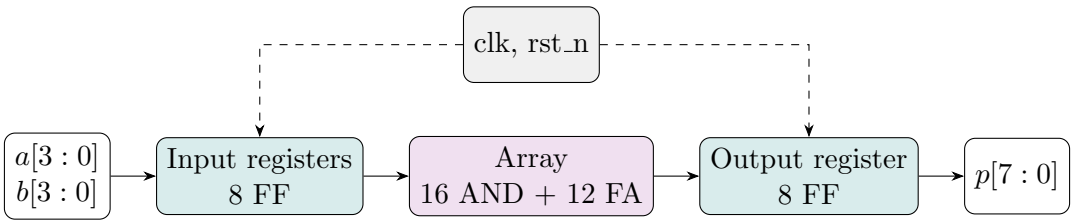


Figure 1: Top-level structure: input registers (8 FF), combinational array (16 AND gates, 12 full adders) and output register (8 FF), 16 flip-flops in total.

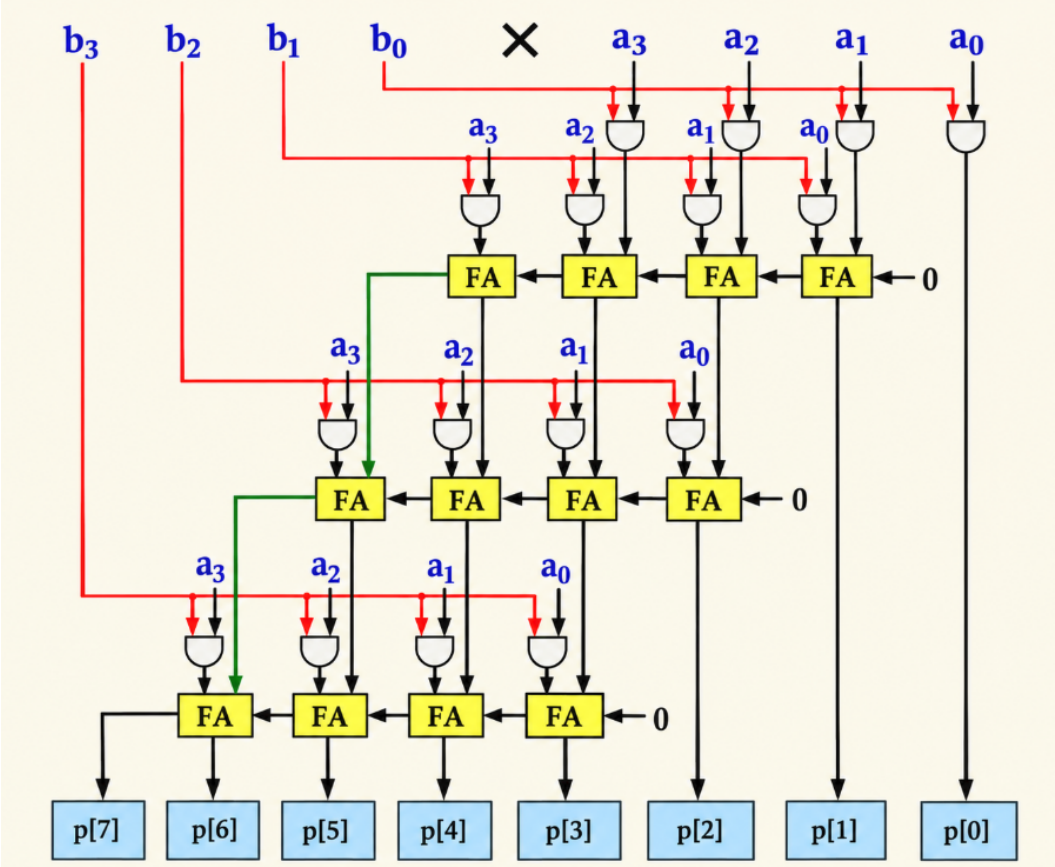


Figure 2: Array structure: 16 AND gates and 12 full adders (FA). Inputs and outputs of this block are registered as in Fig. 1. Red: b_i lines, green: row carry-out.

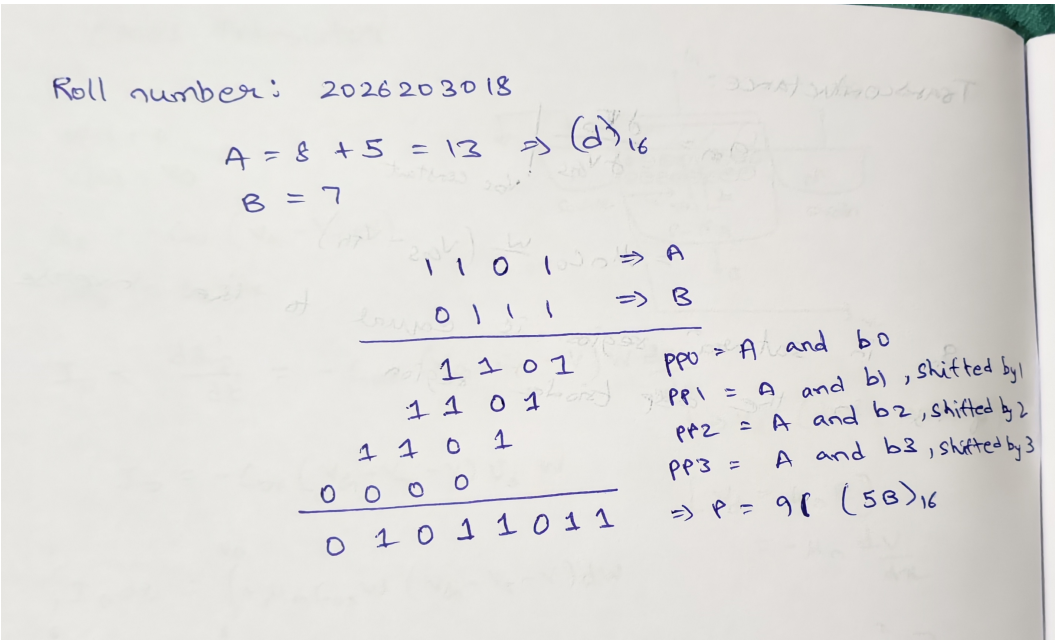


Figure 3: Hand-worked example, roll number 2026203018: $A = 8 + 5 = 13$, $B = 7$, $P = 91 = 0\text{x}5\text{B}$.

Hand example. 13×7: $pp_0 = 1101$, $pp_1 = 11010$, $pp_2 = 110100$, $pp_3 = 0$; the sum is $01011011_2 = 91 = 0\text{x}5\text{B}$.



Figure 4: Waveform for $a = 13$ (hex d). When b changes to 7, p becomes 0x5B (= 91) exactly two rising clock edges (20 ns) later (arrow): latency = 2 cycles. b and p are shown in hex (0x5B = 91).

```
/foss/designs/mult4_2026203018 > vvp sim | tail -n 3
VCD info: dumpfile mult.vcd opened for output.
PASS: 256 vectors checked, 0 errors
tb/tb_mult.v:74: $finish called at 7690000 (1ps)
/foss/designs/mult4_2026203018 >
```

```
/foss/designs/mult4_2026203018 > vvp sim | tail -n 3
MISMATCH: a=15, b=15 | Expected=225, Got=181
FAIL: 256 vectors checked, 95 errors
tb/tb_mult.v:72: $finish called at 5137000 (1ps)
/foss/designs/mult4_2026203018 >
```

Figure 5: Top: correct design, `PASS: 256 vectors checked, 0 errors`. Bottom: error test with `sum = a ^ b` in `full_adder.v`: `FAIL: 256 vectors checked, 95 errors`, so the testbench detects a broken adder. (The "Expected" value in the mismatch line is printed truncated to its low 4 bits: 195 → 3, 210 → 2, 225 → 1.)

3 Synthesis (Part B)

`synth/synth.tcl` reads the RTL, runs `synth -top mult_array -flatten`, maps flip-flops (`dfflibmap`) and logic (`abc`) to SKY130, cleans up and writes the statistics report and the gate-level netlist.

Table 1: Part B results (from `mult_array_stat.rpt`).

Metric	Value
Total number of cells	65
Number of flip-flops	16 (`sky130_fd_sc_hd__dfrtp_1`)
Total cell area	746.966 μm^2
Flip-flop area	400.384 μm^2 (16 × 25.024 μm^2)
Flip-flop area (% of total)	53.60%
3 most-used cell types	`dfrtp_1`: 16, `nand2_1`: 12, `xnor2_1`: 5
Netlist simulation result	`PASS: 256 vectors checked, 0 errors`

Netlist simulation. The command was executed as follows:

```
iverilog -g2012 -DFUNCTIONAL -DUNIT_DELAY=#1 -o gls
tb/tb_mult.v $CELLS/primitives.v
$CELLS/sky130_fd_sc_hd.v synth/mult_array_netlist.v
```

```
/foss/designs/mult4_2026203018 > vvp gls | tail -n 3
VCD info: dumpfile mult.vcd opened for output.
PASS: 256 vectors checked, 0 errors
tb/tb_mult.v:72: $finish called at 5137000 (1ps)
```

Figure 6: Gate-level simulation

4 Timing (Part C)

`sta/sta.tcl` loads the typical-corner library (`tt_025C_1v80`), reads and links the netlist, defines `clk` (period T), 1.0 ns input and output delays, and reports the worst slack and the critical path. At $T = 10$ ns the worst slack was 7.43 ns, so $T_{min} = 10 - 7.43 = 2.57$ ns and $F_{max} = 1000/2.57 \approx 389.1$ MHz. Re-running at $T = 2.57$ ns gave a slack of 0.004 ns (met).

Table 2: Part C results (typical corner).

Metric	Value
Worst slack at $T = 10$ ns	7.43 ns
T_{min}	2.57 ns
F_{max}	389.1 MHz
Slack at $T = T_{min}$	0.004 ns
Critical path start point	`_102_` = `b_reg[0]` (`dfrtp_1`)
Critical path end point	`_096_` = `p[6]` register (`dfrtp_1`)
Number of cells on critical path	8 logic cells (10 including the two flip-flops)
Latency $= 2 \times T_{min}$	5.14 ns
Throughput	389.1 million results per second

Table 3: Critical path `b_reg[0]` $\rightarrow$ `p[6]` at $T = 2.57$ ns (arrival 2.432 ns, required 2.436 ns).

#	Cell	Delay (ns)	Arrival (ns)
launch	`dfrtp_1` (`_102_`, clk$\rightarrow$Q)	0.387	0.387
1	`and2_0`	0.162	0.549
2	`or3_1`	0.391	0.940
3	`and4_1`	0.230	1.170
4	`nor3_1`	0.210	1.380
5	`o21ai_0`	0.131	1.511
6	`maj3_1`	0.386	1.898
7	`maj3_1`	0.359	2.257
8	`xnor3_1`	0.175	2.432
capture	`dfrtp_1` (`_096_`), setup 0.134		slack 0.004

Three of the eight logic cells are adder-type cells (two `maj3` carry cells and one `xnor3` sum cell). The other five (`and2`, `or3`, `and4`, `nor3`, `o21ai`) are partial-product and merged logic.

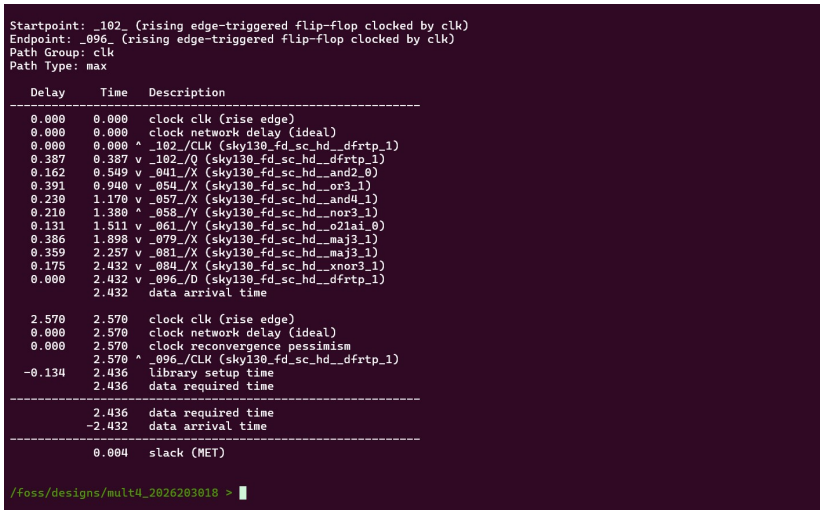


Figure 7: Critical-path report at $T = 2.57$ ns (`mult_array_timing.rpt`).

5 Physical Design (Part D)

`pnr/config.json` sets `DESIGN_NAME` `mult_array`, the two Verilog files, `CLOCK_PORT` `clk`, `FP_SIZING` `absolute` and `DIE_AREA` $[0, 0, 150, 150]$. The starting clock period was $1.2 \times 2.57 = 3.084 \rightarrow 3.5$ ns. LibreLane also checks setup at slow and fast corners, so the first runs failed at `ss_100C_1v60`.

Table 4: LibreLane run log.

Run	`CLOCK_PERIOD`	Worst setup slack	Result
1	3.5 ns	-2.133 ns (`ss_100C_1v60`)	failed (setup)
2	6.0 ns	-0.119 ns (`ss_100C_1v60`)	failed (setup)
3	6.5 ns	-0.019 ns (`ss_100C_1v60`)	failed (setup)
4	7.0 ns	$+0.427$ ns (`ss_100C_1v60`)	passed, DRC 0, LVS 0

Table 5: Part D results (final run, 7.0 ns).

Metric	Value
`CLOCK_PERIOD` used	7.0 ns
Number of runs needed	4
Core area	17759.5 μm^2 (die 22500 μm^2 = 150 × 150)
Cells after synthesis (Part B)	65
Cells after place and route	1885 total (328 excluding fill cells)
Kinds of cells added	tap cells 245, fill cells 1557, timing-repair buffers 19, clock buffers 3
Wirelength	2314 μm (routed)
Setup slack	+0.427 ns (worst, `ss_100C_1v60`); +3.80 ns typical corner
Post-route $F_{max} = 1000/(T -$ slack)	152.1 MHz (slow corner); 312.4 MHz (typical corner)
F_{max} from Part C	389.1 MHz
DRC errors	0 (route, Magic and KLayout all 0; antenna passed)
LVS errors	0

The 1885 instances break down as 16 flip-flops, 45 logic cells, 245 tap, 1557 fill, 19 timing-repair buffers and 3 clock buffers. LibreLane runs its own synthesis, so its 61 non-physical cells (16 + 45) differ slightly from the 65 of Part B.

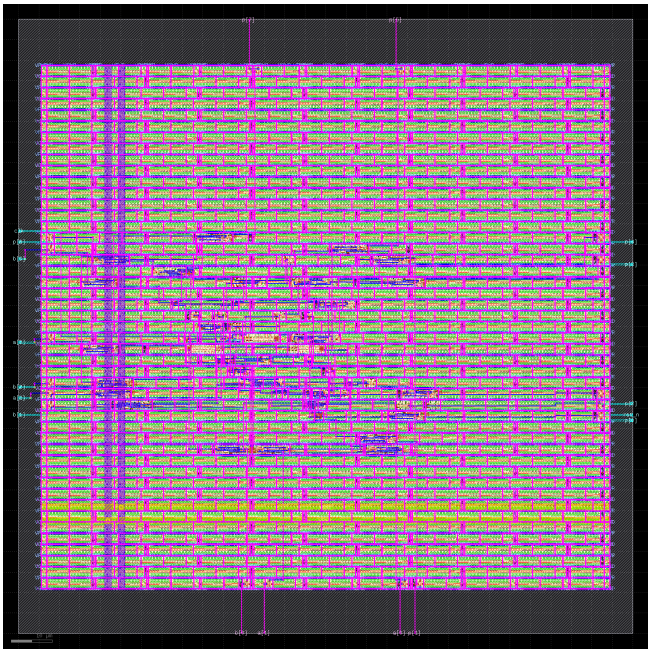


Figure 8: Whole die, 150 μm × 150 μm (KLayout).

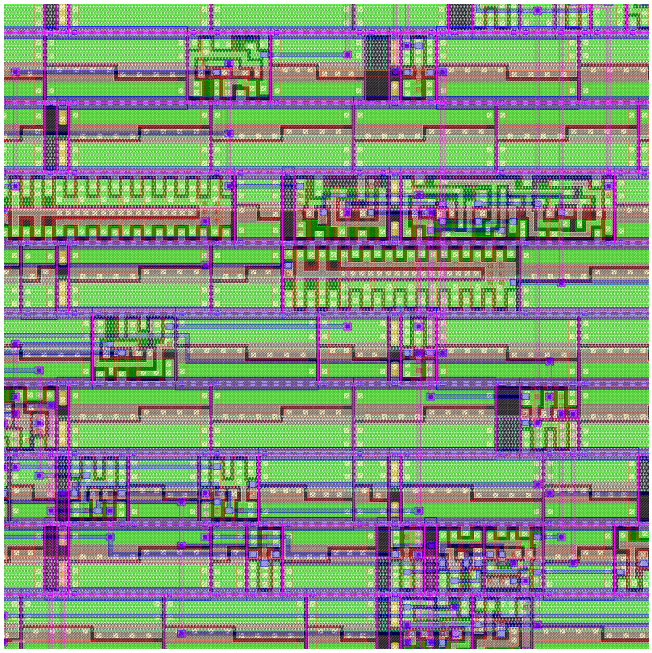


Figure 9: Zoomed view (40 μm × 40 μm) showing standard-cell rows and metal wires.

6 Answers to the Questions

1. Logic cells. Synthesis gave 65 − 16 = 49 logic cells, not 16 + 12 = 28. The 28 counts RTL gates, but synthesis flattens the design and re-maps all the logic onto library gates. Each full

adder needs separate sum and carry cells, so a single adder is more than one cell. Partial-product ANDs are merged with adder logic (12 `nand2_1`), the first adder of each row has carry-in 0 and shrinks to a half adder, and ABC adds inverters and polarity cells.

2. Critical path. The path runs from the input register `b_reg[0]` to the output register `p[6]`. It has 8 logic cells (10 with the two flip-flops) and 2.045 ns of logic delay, which gives $T_{min} = 0.387 + 2.045 + 0.134 = 2.566 \approx 2.57$ ns (clk→Q + logic + setup). The estimate is about $2N = 8$ full-adder delays. The cell count (8) matches, but only 3 of the 8 cells are adder cells (two `maj3`, one `xnor3`), and the average delay is only 0.26 ns per cell. Synthesis has restructured the ripple chain into fewer, faster cells, so the real path is shorter than eight full adders.

3. Flip-flop area. Flip-flops take 400.384/746.966 = 53.6% of the area. One `dfrtp_1` is 25.024 μm^2, about 6.7× a `nand2_1` (3.754 μm^2). The design has 16 flip-flops, because all inputs and the output are registered, but only 49 small logic cells (346.6 μm^2 in total), so the registers dominate even in a small design.

4. Post-route F_{max}. After routing, F_{max} is 312.4 MHz at the typical corner and 152.1 MHz at the slow corner, against 389.1 MHz in Part C. Part C used ideal clocks and no wires. The layout adds wire delay (2314 μm of wire), a real clock tree and repair buffers, the target was a relaxed 7.0 ns, and signoff also checks a slow corner (100 °C, 1.60 V). The flow added four kinds of cells: 245 *tap cells* connect the wells to power and ground and prevent latch-up; 1557 *fill cells* fill empty row space so the wells, implants and power rails are continuous and density rules are met; 19 *timing-repair buffers* fix slew, load and setup problems; 3 *clock buffers* deliver the clock to all 16 flip-flops at nearly the same time. No separate decap cells appear in the metrics.

5. DRC and LVS. DRC checks the layout geometry against the foundry's manufacturing rules (minimum width, spacing, enclosure, density, antenna rules). LVS extracts a netlist from the layout and checks that its devices and connections match the intended netlist. Both are needed: a DRC failure means the layout may not be manufacturable or may give poor yield, and an LVS failure means the chip is not the circuit that was designed. This design has 0 DRC and 0 LVS errors.

6. 8-bit multiplier. For $N = 8$: AND gates $N^2 = 64$ (from 16) and full adders $N(N-1) = 56$ (from 12, about 4.7× more). The critical path is about $2N = 16$ full-adder delays instead of 8, so it doubles; delay grows linearly while area grows quadratically. Scaling the measured logic delay, $2 \times 2.045 = 4.09$ ns, so $T_{min} \approx 0.387 + 4.09 + 0.134 \approx 4.6$ ns and $F_{max} \approx 217$ MHz (typical corner, assuming the same per-stage delay).

7 Conclusion

The multiplier passes all 256 vectors in RTL and as a gate-level netlist (the error test confirms the testbench catches faults), reaches $T_{min} = 2.57$ ns ($F_{max} \approx 389$ MHz, typical corner) before layout, and produces a DRC/LVS-clean layout at 7.0 ns with +0.427 ns worst slack across all corners. Registers dominate the area (53.6%), and the physical flow adds mainly tap and fill cells, with wire and corner effects lowering the post-route F_{max}.

Appendix

Supporting Logs

```
7. Printing statistics.

=== mult_array ===

        +----------Local Count, excluding submodules.
        |        +-Local Area, excluding submodules.
        |        |
       51        - wires
       81        - wire bits
       10        - public wires
       40        - public wire bits
        5        - ports
       18        - port bits
       65  746.966 cells
        1    7.507   sky130_fd_sc_hd__a21boi_0
        3   15.014   sky130_fd_sc_hd__a21oi_1
        1    7.507   sky130_fd_sc_hd__a22oi_1
        1    8.758   sky130_fd_sc_hd__a32oi_1
        2   12.512   sky130_fd_sc_hd__and2_0
        1    8.758   sky130_fd_sc_hd__and4_1
        2    7.507   sky130_fd_sc_hd__clkinv_1
       16  400.384   sky130_fd_sc_hd__dfrtp_1
        1    6.256   sky130_fd_sc_hd__lpflow_inputiso1p_1
        2   12.512   sky130_fd_sc_hd__lpflow_isobufsrc_1
        3   30.029   sky130_fd_sc_hd__maj3_1
       12   85.043   sky130_fd_sc_hd__nand2_1
        2   12.512   sky130_fd_sc_hd__nand4_1
        1    3.754   sky130_fd_sc_hd__nor2_1
        2   10.01    sky130_fd_sc_hd__nor3_1
        2   15.014   sky130_fd_sc_hd__o21a_1
        2   10.01    sky130_fd_sc_hd__o21ai_0
        1    7.507   sky130_fd_sc_hd__a21bai_1
        1    6.256   sky130_fd_sc_hd__or3_1
        5   43.792   sky130_fd_sc_hd__xnor2_1
        3   67.565   sky130_fd_sc_hd__xnor3_1
        1    8.758   sky130_fd_sc_hd__xor2_1

   Chip area for module '\mult_array': 746.966400
     of which used for sequential elements: 400.384000 (53.60%)

/foss/designs/mult4_2026203018 >
```

(a) Yosys statistics report (`mult_array_stat.rpt`).

```
/foss/designs/mult4_2026203018 > grep -i slack sta/mult_array_timing.rpt
           1.798   slack (MET)
           0.004   slack (MET)
/foss/designs/mult4_2026203018 >
```

(e) Slack report: reset recovery (1.798 ns) and critical path (0.004 ns).

```
/foss/designs/mult4_2026203018 > grep -E "Startpoint|Endpoint|Path Group|sky130_fd_sc_hd__|slack" sta/mult_array_timing.rpt
Startpoint: rst_n (input port clocked by clk)
Endpoint: _090_ (recovery check against rising-edge clock clk)
Path Group: asynchronous
   0.000    1.000 ^ _090_/RESET_B (sky130_fd_sc_hd__dfrtp_1)
            2.570 ^ _090_/CLK (sky130_fd_sc_hd__dfrtp_1)
            1.798   slack (MET)
Startpoint: _102_ (rising edge-triggered flip-flop clocked by clk)
Endpoint: _096_ (rising edge-triggered flip-flop clocked by clk)
Path Group: clk
   0.000    0.000 ^ _102_/CLK (sky130_fd_sc_hd__dfrtp_1)
   0.387    0.387 v _102_/Q (sky130_fd_sc_hd__dfrtp_1)
   0.162    0.549 v _041_/X (sky130_fd_sc_hd__and2_0)
   0.391    0.940 v _054_/X (sky130_fd_sc_hd__or3_1)
   0.230    1.170 v _057_/X (sky130_fd_sc_hd__and4_1)
   0.210    1.380 ^ _058_/Y (sky130_fd_sc_hd__nor3_1)
   0.131    1.511 v _061_/Y (sky130_fd_sc_hd__o21ai_0)
   0.386    1.898 v _079_/X (sky130_fd_sc_hd__maj3_1)
   0.359    2.257 v _081_/X (sky130_fd_sc_hd__maj3_1)
   0.175    2.432 v _084_/X (sky130_fd_sc_hd__xnor3_1)
   0.000    2.432 v _096_/D (sky130_fd_sc_hd__dfrtp_1)
            2.570 ^ _096_/CLK (sky130_fd_sc_hd__dfrtp_1)
            0.004   slack (MET)
/foss/designs/mult4_2026203018 > grep -A6 "dfrtp_1 _102_" synth/mult_array_netlist.v
  sky130_fd_sc_hd__dfrtp_1 _102_ (
    .CLK(clk),
    .D(b[0]),
    .Q(b_reg[0]),
    .RESET_B(rst_n)
  );
  sky130_fd_sc_hd__dfrtp_1 _103_ (
/foss/designs/mult4_2026203018 > grep -A6 "dfrtp_1 _096_" synth/mult_array_netlist.v
  sky130_fd_sc_hd__dfrtp_1 _096_ (
    .CLK(clk),
    .D(p_comb[6]),
    .Q(p[6]),
    .RESET_B(rst_n)
  );
  sky130_fd_sc_hd__dfrtp_1 _097_ (
/foss/designs/mult4_2026203018 >
```

(b) Critical-path cells, netlist entries showing `_102_` = `b_reg[0]` and `_096_` = `p[6]`.

```
/foss/designs/mult4_2026203018 > sta sta/sta.tcl
OpenSTA 3.1.0 f89887b596 Copyright (c) 2026, Parallax Software, Inc.
License GPLv3: GNU GPL version 3 <http://gnu.org/licenses/gpl.html>

This is free software, and you are free to change and redistribute it
under certain conditions; type `show_copying' for details.
This program comes with ABSOLUTELY NO WARRANTY; for details type `show_warranty'.
worst slack max 7.43
/foss/designs/mult4_2026203018 >
```

(c) Worst slack at $T = 10$ ns (7.43 ns).

```
/foss/designs/mult4_2026203018 > T=2.57 sta sta/sta.tcl
OpenSTA 3.1.0 f89887b596 Copyright (c) 2026, Parallax Software, Inc.
License GPLv3: GNU GPL version 3 <http://gnu.org/licenses/gpl.html>

This is free software, and you are free to change and redistribute it
under certain conditions; type `show_copying' for details.
This program comes with ABSOLUTELY NO WARRANTY; for details type `show_warranty'.
worst slack max 0.00
/foss/designs/mult4_2026203018 >
```

(d) Worst slack at $T = 2.57$ ns (0.00 ns).

Figure 10: Supporting logs for Parts B and C.

Individual and Joint Contributions

Ramit Nag (Roll No. 2026203011)

- **Part A — RTL Design:** Developed the 4-bit array multiplier architecture, including the full-adder modules, partial-product generation, and input/output registers.
- **Part A — Verification:** Participated in functional verification, waveform analysis, and checking the two-cycle latency.
- **Part C — Static Timing Analysis:** Performed OpenSTA analysis, identified the critical path, calculated the minimum clock period and maximum operating frequency.
- **Part D — Timing Closure:** Analyzed setup timing violations across different clock periods and evaluated the final post-route timing and performance.
- **Documentation:** Contributed to the design explanation, STA results, performance analysis, and technical report.

Kanuthala Harshith (Roll No. 2026203018)

- **Part A — Verification:** Developed and validated the exhaustive testbench covering 256 input combinations and intentional fault detection.
- **Part B — Logic Synthesis:** Performed synthesis using Yosys, generated the technology-mapped netlist, analyzed cell utilization and area, and verified gate-level functionality.
- **Part D — Physical Design:** Configured and executed the LibreLane flow, including floorplanning, placement, clock-tree synthesis, routing, and GDSII generation.
- **Part D — Physical Verification:** Analyzed DRC/LVS results, inspected the layout using KLayout, and prepared physical-design screenshots.
- **Documentation and Version Control:** Contributed to the final report, README, GitHub repository organization, and submission preparation.

Joint Contributions

Both team members collaborated on debugging, interpreting simulation and timing results, reviewing the RTL-to-GDSII flow, answering the assignment questions, and validating the final deliverables.